

Tesztdokumentáció elkészítése – Exotic Rentals projekt

Ez a dokumentum az Exotic Rentals (autóbérlő és sofőrszolgálat) webalkalmazás tesztelési folyamatait és eredményeit mutatja be. A tesztelés során a backend API végpontokat, valamint a frontend felhasználói felületét vizsgáltuk.

Tesztkörnyezet és Keretrendszerek

- **Kliens (Frontend) környezet:** React.js, böngésző alapú futtatás (localhost:3000)
- **Szerver (Backend) környezet:** ASP.NET Core Web API (localhost:7065), MySQL adatbázis
- **Frontend tesztelési keretrendszer:** Python unittest és Selenium Webdriver (Google Chrome böngészővel) a funkcionális (E2E) teszteléshez.
- **Backend tesztelési keretrendszer:** Swagger UI integrációs teszteléshez és API végpontok manuális hívásához.

1. Backend Integrációs Teszt (Swagger API)

Az alkalmazás backendje egy RESTful API, amelyet a beépített Swagger UI segítségével teszteltünk. Ennek során a szerver és az adatbázis közötti integrációt, valamint a megfelelő JSON válaszok szerkezetét vizsgáltuk.

Tesztelt végpont: GET /api/vehicles?category=Sedan

Cél: Ellenőrizni, hogy a végpont képes-e kategória alapján szűrni a járműveket, és visszaadja-e a megfelelő adatstruktúrát (benne az azonosítóval, árral, specifikációkkal és a képek listájával).

A Swagger felületen tesztelt E-mail/Jelszó hitelesítés, regisztráció és járműlekérdezés végpontjainak listája:

ExoticBackend		
GET	/api/status	
GET	/api/vehicles	
GET	/api/vehicles/{id}	
GET	/api/gallery	
POST	/api/register	
POST	/api/auth/confirm	
GET	/api/auth/me/{id}	
GET	/api/user/{id}/profile	
PUT	/api/user/{id}/profile	
POST	/api/login	
GET	/api/user/{id}/orders	
POST	/api/orders	
POST	/api/orders/verify	
GET	/api/drivers	
POST	/api/orders/taxi	
POST	/api/orders/taxi/{id}/accept	
GET	/api/driver/{driverId}/taxi-orders	
POST	/api/orders/taxi/{id}/finish	
GET	/api/user/{userId}/taxi-orders	
POST	/api/orders/{id}/finish	
POST	/api/users/{id}/upload-pfp	
GET	/api/users/{id}/profile	
POST	/api/user/{id}/verify-license	
POST	/api/auth/forgot-password	
POST	/api/auth/reset-password	
POST	/api/user/{id}/change-password	

A végrehajtott kérés (cURL) és a kapott válasz (Response body):

Curl

```
curl -X 'GET' \
  'https://localhost:7065/api/vehicles?category=Sedan' \
  -H 'accept: */*'
```

Request URL

```
https://localhost:7065/api/vehicles?category=Sedan
```

Server response

Code	Details
200	Response body <pre>{ "id": 1, "brand": "Mercedes-Benz", "model": "S63 AMG E-Performance", "category": "Sedan", "drivetrain": "AWD", "transmission": "9-Fokozatú Automata", "engineType": "4.0L V8 Hibrid", "fuel": "Hibrid", "status": 1, "year": 2026, "pricePerDay": 0, "times_Rented": 12, "images": [{ "id": 1, "imageUrl": "https://www.auto-data.net/images/f38/Mercedes-Benz-EQE-V295.jpg", "isPrimary": true }, { "id": 2, "imageUrl": "https://www.auto-data.net/images/f34/Mercedes-Benz-EQE-V295.jpg", "isPrimary": false }, { "id": 3, "imageUrl": "https://www.auto-data.net/images/f53/Mercedes-Benz-EQE-V295.jpg", "isPrimary": false }] }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 17 Apr 2026 17:23:30 GMT server: Kestrel</pre>

Eredmény: A teszt sikeres. A szerver 200 OK HTTP státuszkóddal tért vissza, és a JSON válaszban hibátlanul megjelent a szűrésnek megfelelő "Mercedes-Benz S63 AMG E-Performance" jármű az összes relációs adattal (képekkel) együtt.

2. Frontend Funkcionális Tesztelés (Selenium Webdriver)

A frontend felhasználói élményének (UI/UX) és működésének ellenőrzésére automatizált funkcionális tesztekert írtunk Python nyelven, a Selenium Webdriver használatával. A teszt a Chrome böngészőt automatizálva szimulálja egy valós felhasználó lépéseit

A teszt kód (Test_Exotic_Frontend.py) 1. kép:

```
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

class ExoticRentalsTests(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        print("Böngésző elindítása...")
        cls.driver = webdriver.Chrome()
        cls.driver.maximize_window()
        cls.BASE_URL = "http://localhost:3000"

    @classmethod
    def tearDownClass(cls):
        print("Teszt befejeződtek. Böngésző bezárása...")
        time.sleep(2)
        cls.driver.quit()

    def test_1_successful_login(self):
        """1. Teszt: Bejelentkezési folyamat tesztelése a Navbaron keresztül"""
        driver = self.driver
        driver.get(self.BASE_URL)
        wait = WebDriverWait(driver, 10)

        login_dropdown_btn = wait.until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Bejelentkezés')]"))
        )
        login_dropdown_btn.click()

        email_input = wait.until(EC.visibility_of_element_located((By.XPATH, "//input[@type='email']")))
        password_input = driver.find_element(By.XPATH, "//input[@type='password']")

        email_input.send_keys("gamelife9222@gmail.com")
        password_input.send_keys("admin1")

        submit_btn = driver.find_element(By.XPATH, "//button[@type='submit' and contains(text(), 'Bejelentkezés')]")
        submit_btn.click()

        print("Várakozás a bejelentkezésre...")
```

A teszt kód (Test_Exotic_Frontend.py) 2. kép:

```
print("Várakozás a bejelentkezésre...")

profile_btn = wait.until(
    EC.visibility_of_element_located((By.XPATH, "//button[contains(text(), 'Profil')]]"))
)
self.assertTrue(profile_btn.is_displayed(), "A bejelentkezés nem sikerült!")
print("1. Teszt SIKERES: Bejelentkezve maradunk a következő teszthez!")

def test_2_navigation_to_gallery(self):
    """2. Teszt: Routing és navigáció ellenőrzése az oldalsávból"""
    driver = self.driver
    wait = WebDriverWait(driver, 10)

    hamburger_icon = wait.until(
        EC.element_to_be_clickable((By.CSS_SELECTOR, ".hamburger-icon"))
    )
    hamburger_icon.click()

    gallery_link = wait.until(
        EC.element_to_be_clickable((By.XPATH, "//a[contains(@class, 'sidebar-item') and contains(text(), 'Galéria')]]"))
    )
    gallery_link.click()

    print("Navigálás a Galéria oldalra...")

    current_url = driver.current_url
    self.assertIn("/Gallery", current_url, "Az URL nem változott meg a /Gallery végződésre!")

    gallery_title = wait.until(
        EC.visibility_of_element_located((By.XPATH, "//h1[contains(@class, 'gallery-hero-title')]]"))
    )
    self.assertEqual(gallery_title.text, "Galéria", "A Galéria oldal főcíme nem töltött be!")

    print("2. Teszt SIKERES: A navigáció és a Routing hibátlanul működik!")

if __name__ == "__main__":
    unittest.main()
```

Automatizált teszt futás eredménye:

```
end/frontend/src/Tests/Test_Exotic_Frontend.py
Böngésző elindítása...
Várakozás a bejelentkezésre...
1. Teszt SIKERES: Bejelentkezve maradunk a következő teszthez!
.Navigálás a Galéria oldalra...
2. Teszt SIKERES: A navigáció és a Routing hibátlanul működik!
.Teszttek befejeződtek. Böngésző bezárása...
```

```
-----
Ran 2 tests in 16.668s
```

OK

Az automatizált teszt sikeresen lefutott. A tesztelő script elindította a böngészőt, sikeresen bejelentkezett a felhasználói fiókba, majd hibátlanul navigált át a Galéria menüpontba. Mindkét tesztet (a validációs elvárásokkal együtt) összesen 16.668 másodperc alatt hiba nélkül (OK) lezajlott.

A teszteket készítette:

Futó Ábel (Backend Tesztek)

Márton Roland (Frontend Tesztek)